Cooper Martin
V00969649
Csc226

Minimum Cost Path Investigation

Conjecture

Even for relatively small values of k, the Floyd-Warshall-Roy algorithm will provide better performance in practice for the k-source minimum cost path problem than k runs of Dijkstra's algorithm (regardless of the asymptotic running times).

Definition of Better Performance

Primary Definition

For a fixed direction & edge-weight graph $G = (V, E)$ with no negative edge costs and a fixed and source set $S = \{s_1, \ldots, s_k\}$.

Let

1. $T_{\text{Djisktra}} = T_{\text{D}}(G, S)$ be the median elapsed time required to run Dijkstra's algorithm on G once for each source vertex $s \in S$.
2. $T_{\text{Floyd-Warshall}} = T_{\text{FW}}(G)$ be the median elapsed time required to run Floyd-Warshall once on G. For a problem instance on (G,S)

If

1. $T_{\text{FW}}(G) < T_{\text{D}}(G, S)$, then Floyd-Warshall performs better on the problem instance
2. $T_{\text{D}}(G, S) < T_{\text{FW}}(G)$, then Repeated Dijkstra performs better on the problem instance

All other computed metrics exist to aid in explaining why one method wins or loses in a given case. My implementation focuses on computing distance arrays rather than explicitly reconstructing predecessor/shortest path trees, so the timed region of the test only covers shortest-path distance computations and does not include any runtime eaten up by setup operations. This is consistent with the conjecture's emphasis on a practical evaluation of performance.

Secondary Metrics

The two primary values $T_{\mathrm{FW}}(G)$ and $T_{\mathrm{D}}(G, S)$ are sufficient to evaluate the conjecture, yet they are not descriptive enough to provide the necessary context for different graph cases, so some secondary quantities are recorded to aid in the interpretation of the results.

Actual graph density

The input parameter edge_density controls how likely edges are to be generated. Differing effects based on which of the two graph families (random or local) the graph was generated with, so the same input edge_density produces different actual graph densities depending on graph family. Actual density is computed as

$$\text{actual_density} = \frac{|E|}{n(n-1)} = \frac{|E|}{|V|(|V|-1)},$$

Where $|E|$ is total number of edges in G, n = $|V|$ is the total number of vertices in G, and n(n-1) represents the total number of possible directed edges in a graph of n = $|V|$ vertices with no self loops. This gives the true fraction of possible edges present in the generated graph, regardless of which family produced it.

Runtime ratio

The ratio $R(G, S) = T_{\mathrm{FW}}(G)/T_{\mathrm{D}}(G, S)$ normalizes the comparison onto a single interpretable scale.

1. If $R(G, S) < 1$ Floyd-Warshall wins because $T_{\mathrm{FW}}(G) < T_{\mathrm{D}}(G, S)$
2. If $R(G, S) > 1$ k repeated Dijkstra wins because $T_{\mathrm{D}}(G, S) < T_{\mathrm{FW}}(G)$

The magnitude of $R(G, S)$ shows the factor by which one method outperformed the other, for example, R = 100 means Floyd-Warshall was one hundred times as long as repeated Dijkstra on a given problem instance.

Per-source Dijkstra time

The ratio $T_{\mathrm{D}}(G, S)/k$ is the median Dijkstra time divided by the number of sources, which separates total batch cost from per source cost. This makes it possible to see if each individual Dijkstra run is getting faster or slower as k changes independently of batch size.

Methodology

Algorithm Implementations

Both algorithms were implemented in python. I adapted from Geeks4Geeks implementations and made two modifications: Floyd-Warshall uses INF rather than -1 to represent no-edge and operates on a copy of the input matrix rather than modifying it in place. Dijkstra uses a min-heap priority queue with the same INF convention.

Graph Generation

Graphs are generated as edge lists and converted into the necessary input format required by each algorithm. An edge list is converted into an adjacency list for Dijkstra and an adjacency matrix for Floyd-Warshall. Two graph families were used:

1. Random: Each edge is generated by including each possible directed edge independently with probability edge_density, so actual density is closer to the input parameter.
2. Local: Each vertex restricted to edges within a neighbourhood of size $n/8$, causing actual density to be lower than the input parameter, regardless of edge_density.

Both random and local use a hamiltonian cycle to ensure all vertices are reachable, with integer edge weights being uniformly drawn from 1 to 10. The tested parameter configuration space was:

- Number of sources: $k \in \{1, 2, 4, 8\}$
- Number of vertices: $n \in \{20, 40, 80, 120\}$
- Graph families: family $\in \{\text{random}, \text{local}\}$
- Edge densities: edge_density $\in \{0.10, 0.25, 0.50\}$

Timing

The timed region for repeated Dijkstra begins immediately before the batch of k single-source runs and ends immediately after the final run completes. The timed region for Floyd-Warshall begins immediately before the all-pairs computation and ends immediately after it returns. All preliminary work including graph generation, representation conversion, source selection, printing, csv output are excluded. For each test case, one warmup run is performed and discarded, followed by seven measured runs. The median time of those runs is then used as the primary statistic. Median was chosen over mean to reduce timing noise from the execution environment

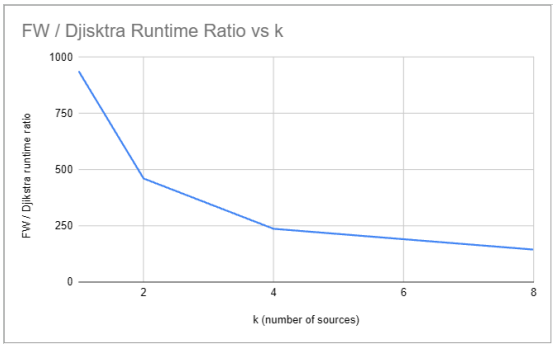
Correctness Verification

After each test case, for every source s ∈ S, the distance array returned by Dijkstra is compared row-by-row against row s of the Floyd-Warshall distance matrix. All 288 test cases returned match = True, confirming both implementations solve the same problem correctly and that the runtime comparison is meaningful.

Results

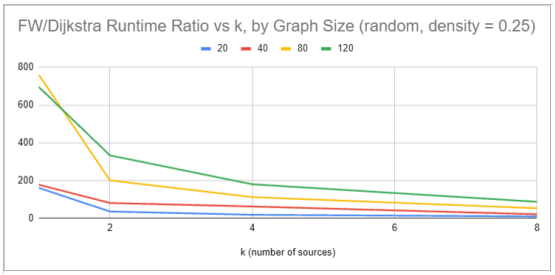
FW/Djisktra Runtime Ratio vs k

Across all tested graph sizes, families, and densities, the FW/Dijkstra runtime ratio decreased monotonically as k increased from 1 to 8, falling from a mean of 939 at k=1 to 144 at k=8. The ratio remained far above 1 for every value of k, so repeated Dijkstra was faster in every case. By my definition of better performance, the method with the smaller median elapsed solve time, Floyd-Warshall never outperformed repeated Dijkstra for any tested value of k. See appendix for table.



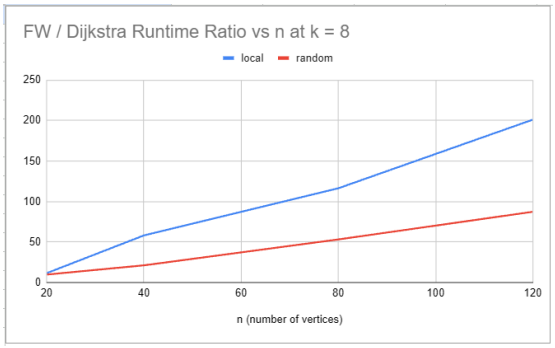
FW/Dijkstra Runtime Ratio vs k, by Graph Size (random, density=0.25)

Broken down by graph size, larger n produces a higher ratio at every value of k. At k=8, the ratio for n=20 is approximately 10, for n=120 it's approximately 87. This is opposite of what the conjecture requires: as n grows, Floyd-Warshall becomes relatively more expensive compared to repeated Dijkstra, not more competitive. The $O(n^3)$ cost of Floyd-Warshall dominates increasingly as n increases, while k Dijkstra runs on sparse graphs scale much more slowly. See appendix for table.



FW/Dijkstra Runtime Ratio vs n, k = 8

At k=8, the most favourable value of k for Floyd-Warshall, both graph families show the ratio increasing with n. The local family consistently produces higher ratios than random at every n. At n=120, local yields a ratio of around 201 vs 87 for random. Local produces graphs with lower actual density than random at the same edge_density, so Dijkstra has less work on local graphs, making Floyd-Warshall worse. See appendix for table.



Overall $n \times k$ summary

This table below aggregates all tested graph families and densities into a single n by k grid. Every cell in the table is far above 1, which confirms the previous findings that k repeated Dijkstra is faster than Floyd-Warshall across the entire test space. The ratio grows with n and shrinks with k, no combination of parameters brings that ratio close to or below 1. The conjecture is not supported by any entry in this table.

	AVG of $R(G,S)$			
n	$k=1$	$k=2$	$k=4$	$k=8$
20	94.99	39.15	24.37	10.96
40	265.70	115.78	69.28	33.10
80	681.23	305.39	180.61	75.55
120	1017.87	527.31	254.62	147.73

Conclusions

Conjecture evaluation

The conjecture is not supported by my investigation and evaluative methods. K repeated Dijkstra performed better than Floyd-Warshall in all of the 288 test cases and across all configurations of graph size, graph family, density, and number of sources. $R(G,S)$ never fell below 1 in any tested graph configuration with the closest it ever reached being R = 9.7 at n = 20, k = 8, with family=random.

Connection to asymptotic analysis

Floyd-Warshall performs $O(n^3)$ operations regardless of k or the number of edges $|E|$ because it has to compute shortest paths between all possible pairs of vertices, even if they are not needed given the k source min-cost problem. K repeated Dijkstra performs $O(k|E|\log n)$ total operations, doing only as much work as the k requested sources require. Given small k and sparse graphs, k repeated Dijkstra runs therefore are cheaper than a single Floyd-Warshall pass, which is reflected by the experiments' results. As k grows, the repeated Dijkstra cost increases while Floyd-Warshall's cost stays fixed, so the ratio $R(G,S)$ shrinks: the two methods get closer together. If k was large enough, a crossover would eventually occur, but the test range $k \in \{1,2,4,8\}$ is far out from that point. For n=120, k would need to approach n before the two methods become comparable in cost, but then the problem could be defined as the all-pairs case wherein Floyd-Warshall would be the choice algorithm. The results from my experiment seem consistent with what asymptotic analysis predicts. Small-k, sparse graph repeated Dijkstra wins by a sizable time and performance margin.

Limitations and scope

With the highest number of vertices being 120 and highest number of sources being 8, the experiment does not reach the conditions where k is large relative to n. The investigation should have therefore included measurements of performance closer to the region where the required number of sources to visit approaches the total number of vertices in the graph. The densest graphs tested have actual density of roughly 0.5, so the case where $|E|$ approaches n^2 and per-source Dijkstra cost becomes significant is not covered. The investigation's validity is also limited to my python implementation, which may not reflect the relative performance conclusions of professional, optimized code. Finally, only elapsed wall clock time is measured, other measurements such as memory usage, cache behaviour, and allocation overhead are not taken into account, so under different evaluation criteria the results could differ.

Appendix

FW / Djisktra Runtime Ratio vs k

k	AVG of `fw_over_djikstra_ratio` $= R(G,S)$
1	939.2104458
2	460.7576646
4	236.8683958
8	144.2842414

FW/Dijkstra Runtime Ratio vs k, by Graph Size (random, density $= 0.25$)

	AVG of $R(G,S)$			
k	$n = 20$	$n = 40$	$n = 80$	$n = 120$
1	161.224747	178.0209291	761.0902594	696.232865
2	35.7422178	81.02778018	200.8503588	333.3767132
4	18.4323591	62.39877789	112.1286014	180.2447209
8	9.748416174	21.16441941	53.17791307	87.35065607

FW / Dijkstra Runtime Ratio vs n at $k = 8$

	AVG of $R(G,S)$	
n (number of vertices)	local	random
20	11.47611891	9.748416174
40	58.13439851	21.16441941
80	116.4250337	53.17791307
120	201.2178268	87.35065607

References

`https://www.geeksforgeeks.org/dsa/floyd-warshall-algorithm-in-python/`

`https://www.geeksforgeeks.org/dsa/dijkstras-shortest-path-algorithm-greedy-algo-7/`